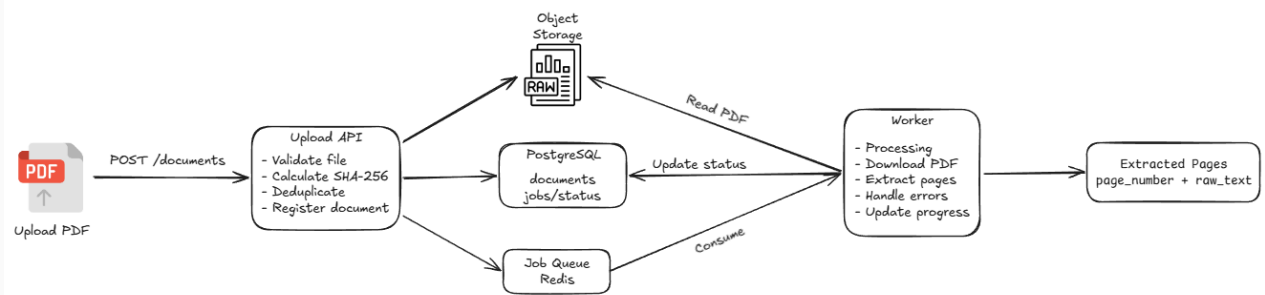


Block 1



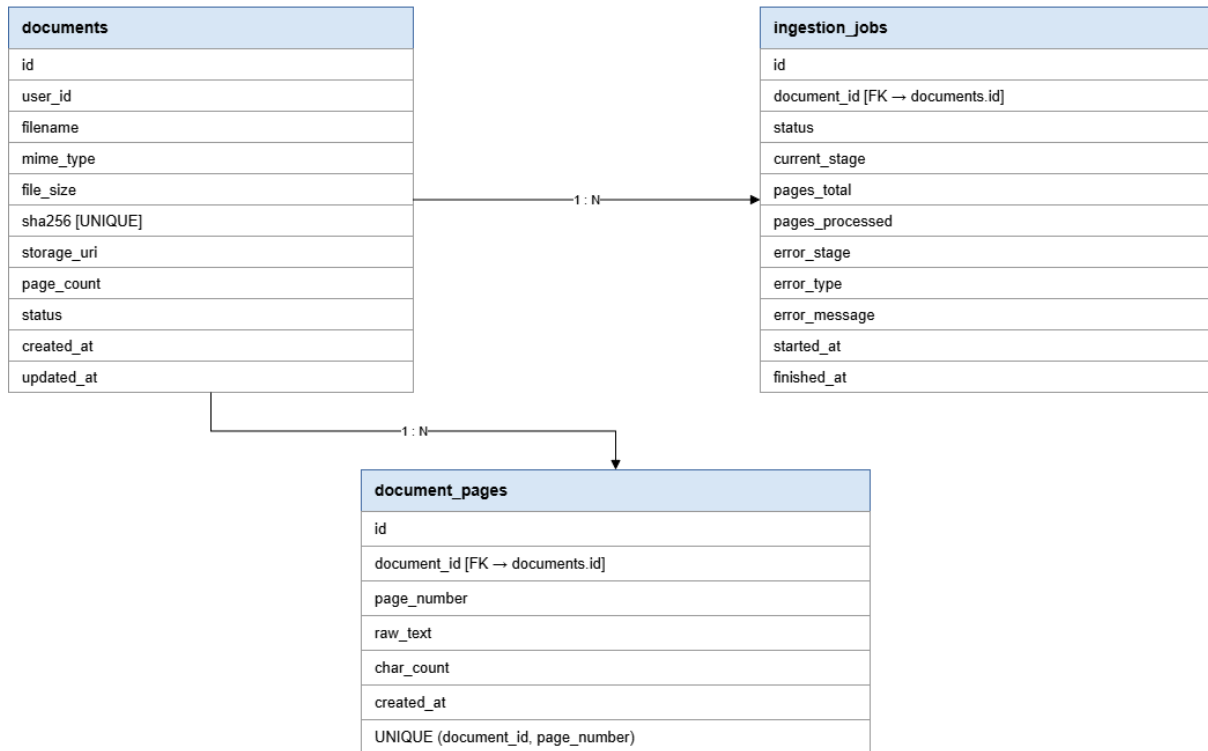
- **Upload PDF / Client:** user gửi file PDF vào hệ thống qua endpoint **POST /documents**. Đây là điểm bắt đầu của pipeline.
- **Upload API:** backend tiếp nhận file và làm các việc đầu tiên như validate file, tính **SHA-256** để phát hiện file trùng, deduplicate, rồi register document vào hệ thống. Upload API là “nhạc trưởng” của giai đoạn tiếp nhận: nó ghi metadata vào DB, lưu file gốc vào Object Storage và tạo job xử lý bất đồng bộ.
- **Object Storage:** nơi lưu **file PDF gốc vật lý**. Trong local stack của ta là MinIO; về concept tương đương S3. Worker sau này đọc lại PDF từ đây để xử lý. PostgreSQL chỉ lưu metadata và **storage_uri**, không lưu binary PDF.
- **PostgreSQL:** nơi lưu **metadata + state của document/job**. Ví dụ document nào đang **UPLOADING, PENDING, PROCESSING, COMPLETED** hay **FAILED**; job đang ở stage nào; đã xử lý bao nhiêu trang; và sau cùng lưu **document_pages**.
- **Job Queue / Redis:** nơi giữ các job đang chờ xử lý. Upload API đẩy một job vào Redis/RQ với thông tin như **job_id, document_id, request_id**. Queue giúp API không phải đứng chờ việc extract PDF hoàn tất.
- **Worker:** background process consume job từ Redis. Khi nhận job, Worker cập nhật trạng thái sang **PROCESSING**, tải PDF từ Object Storage, dùng PyMuPDF để extract text theo từng trang, xử lý retry/error nếu cần, cập nhật progress và persist kết quả vào PostgreSQL.

- **Extracted Pages:** output của Block 1. Mỗi page được biểu diễn tối thiểu bởi:

page_number

raw_text

POSTGRESQL



PostgreSQL — nơi lưu các thông tin có cấu trúc liên quan đến tài liệu và quá trình xử lý

- **documents** để lưu thông tin file (**filename**, **sha256**, **storage_uri**, **status**)
- **ingestion_jobs** để theo dõi job (**PENDING**, **PROCESSING**, **COMPLETED**, **FAILED**, **current_stage**, lỗi, progress)
- **document_pages** để lưu kết quả bóc tách từng trang (**page_number**, **raw_text**, **char_count**)

STORAGE & QUEUE DATA

MINIO — Bucket: documents
{document_id}/
└─ original.pdf
Stable URI
minio://documents/{document_id}/original.pdf

REDIS / RQ — Ingestion Job Payload
job_id
document_id
request_id
retry policy
max = 2
interval = [2s, 5s]

MinIO — nơi lưu file PDF gốc

MinIO là Object Storage, tương tự S3 nhưng chạy local. File PDF thật được lưu theo key ổn định kiểu

`documents/{document_id}/original.pdf`; PostgreSQL chỉ giữ địa chỉ `storage_uri`, không giữ binary PDF. Worker dùng `document_id/storage_uri` để tải đúng file về xử lý.

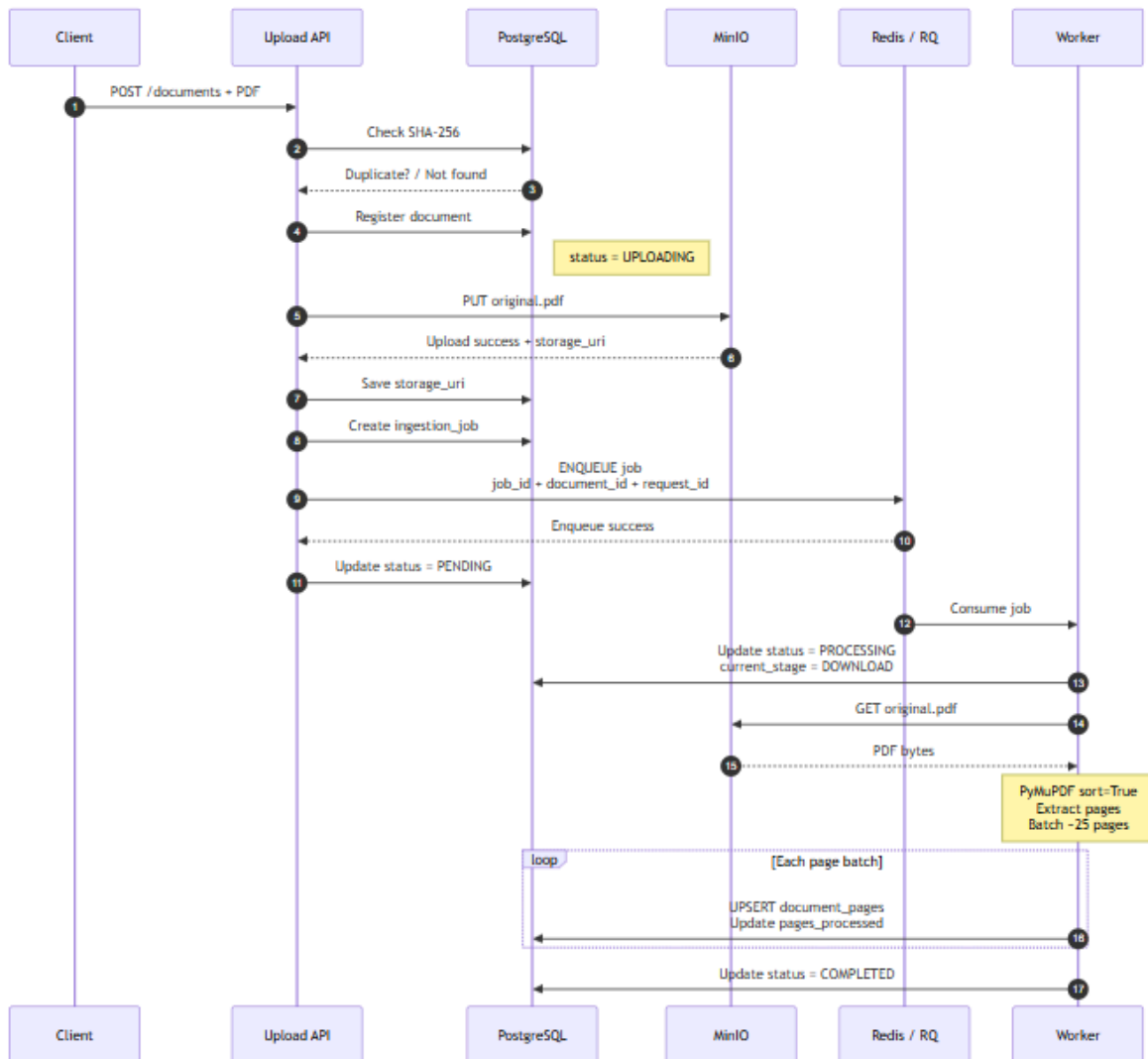
Khi debug, MinIO giúp trả lời: **file gốc có thực sự upload thành công không, file có bị mất/corrupt không, Worker có đang đọc đúng PDF không, SHA-256 của file trong MinIO có khớp file user upload không.**

Redis / RQ — nơi điều phối công việc bất đồng bộ.

Redis giữ queue các job mà Worker cần xử lý. Một job thường mang `job_id, document_id, request_id`; Worker consume job này rồi dùng `document_id` để đọc metadata từ PostgreSQL và tải PDF từ MinIO.

RQ còn quản lý retry khi gặp lỗi tạm thời, hiện tại là retry có giới hạn.

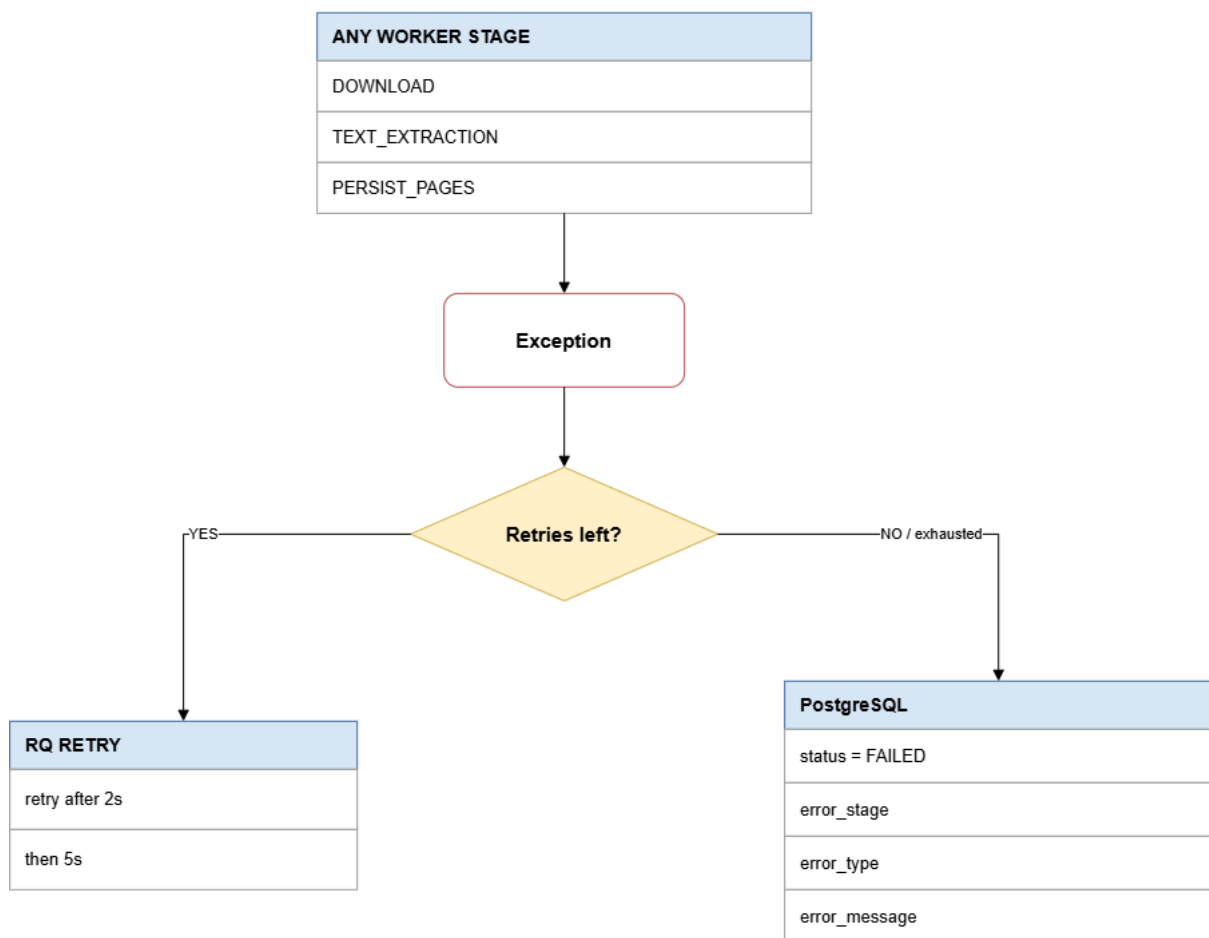
Khi debug, Redis/RQ giúp trả lời: **job đã được enqueue chưa, Worker đã consume chưa, job đang chờ hay retry, có job bị kẹt trong queue không.**



- **Client → Upload API:** user gửi PDF qua **POST /documents**.
- **Upload API → PostgreSQL:** check SHA-256 để phát hiện file trùng, sau đó register document với trạng thái ban đầu **UPLOADING**.
- **Upload API → MinIO:** lưu file PDF gốc.
- **MinIO → Upload API:** trả về việc upload thành công và **storage_uri**.
- **Upload API → PostgreSQL:** lưu **storage_uri**, tạo **ingestion_job**.
- **Upload API → Redis/RQ:** enqueue job chứa **job_id**, **document_id**, **request_id**.
- **Upload API → PostgreSQL:** khi enqueue thành công thì đổi trạng thái thành **PENDING**.
- **Redis/RQ → Worker:** Worker lấy job khỏi queue.

- **Worker → PostgreSQL:** đổi trạng thái thành **PROCESSING**.
- **Worker → MinIO:** yêu cầu file PDF gốc.
- **MinIO → Worker:** trả PDF bytes.
- **Worker xử lý nội bộ** bằng PyMuPDF:
 - PDF*
 - *extract pages*
 - *gom batch ~25 pages*
- **Worker → PostgreSQL:** UPSERT **document_pages**, cập nhật **pages_processed**.
- *status = COMPLETED*

FAILURE / RETRY LANE



Lỗi có thể xảy ra ở các stage như:

- *DOWNLOAD*
- *TEXT_EXTRACTION*

- *PERSIST_PAGES*

Nếu có exception: Exception -> Retries left?

Có hai trường hợp xảy ra.

1. Còn retry

YES -> RQ Retry -> đợi 2s -> thử lại -> nếu tiếp tục lỗi thì đợi 5s

Mục đích là không đánh dấu job **FAILED** ngay với những lỗi tạm thời như mất kết nối ngắn.

2. Hết retry

NO / exhausted -> PostgreSQL

lúc này lưu:

status = FAILED

error_stage

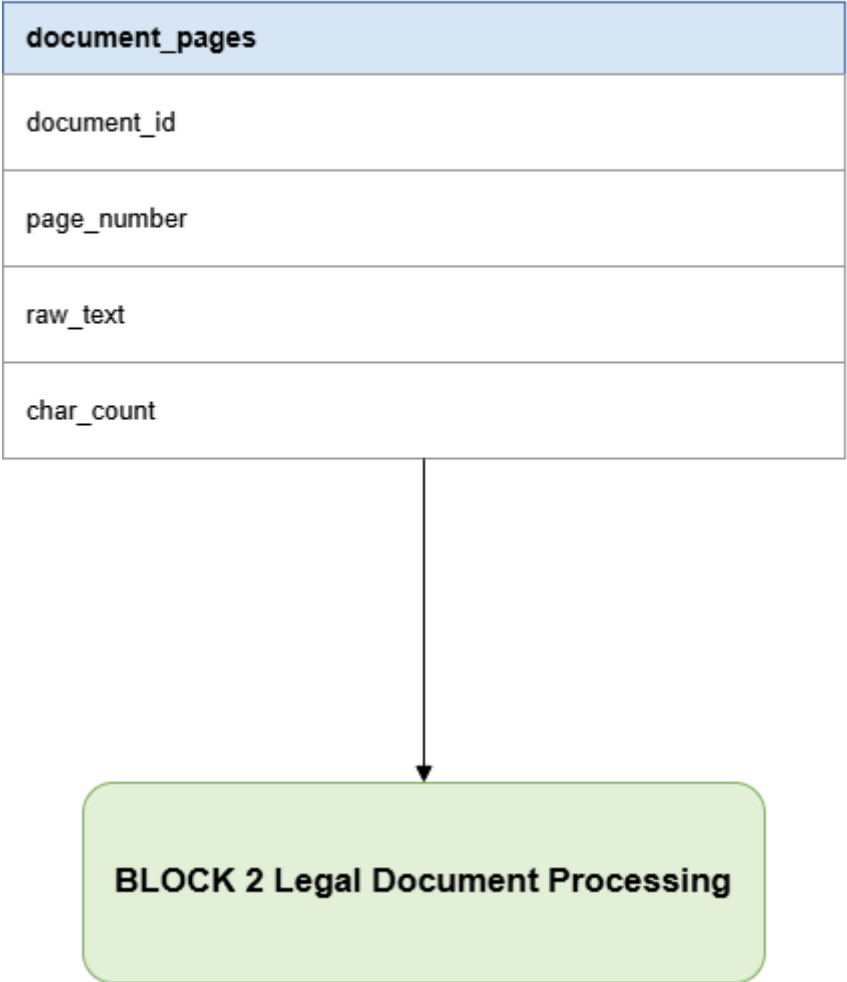
error_type

error_message

Nhờ vậy khi debug ta biết:

- File nào fail?
- Fail ở bước nào?
- Lỗi loại gì?
- Nội dung lỗi là gì?

OUTPUT CONTRACT



Nó chỉ cam kết rằng sau khi hoàn thành sẽ có: *document_pages* với:

document_id
page_number
raw_text
char_count

Block 2 sau này **chỉ cần dựa vào contract này** để bắt đầu:

Nếu Block 2 gặp lỗi: “raw_text trang X bị thiếu” thì quay lại Block 1.

Nếu raw text chuẩn mà: “Điều 5 bị parse sai” thì lỗi thuộc Block 2.